

Module 5: Blockchain 2.0: [6L]

Ethereum and Smart Contracts; The Turing Completeness of Smart Contract Languages and verification challenges; Using smart contracts to enforce legal contracts; comparing Bitcoin scripting vs. Ethereum Smart Contracts.

1. Ethereum and Smart Contracts

Ethereum is a Blockchain network that introduced a built-in Turing-complete programming language that can be used for creating various decentralized applications (also called Dapps). The Ethereum network is fueled by its own cryptocurrency called 'ether'.

- The Ethereum network is currently famous for allowing the implementation of smart contracts. Smart contracts can be thought of as 'cryptographic bank lockers' that contain certain values.
- These cryptographic lockers can only be unlocked when certain conditions are met.
- Unlike bitcoin, Ethereum is a network that can be applied to various other sectors.
- Ethereum is often called Blockchain 2.0 since it proved the potential of blockchain technology beyond the financial sector.
- The consensus mechanism used in Ethereum is Proof of Stakes (PoS), which is more energy efficient when compared to that used in the Bitcoin network, that is, Proof of Work (PoW). PoS depends on the amount of stake a node holds.

Type of Ethereum Accounts

Ethereum has two types of accounts: An externally owned account (EOA), and a Contract account. These are explained as following below:

- **Externally owned account (EOA):** Externally owned accounts are controlled by private keys. Each EOA has a public-private key pair. The users can send messages by creating and signing transactions.
- **Contract Account:** Contract accounts are controlled by contract codes. These codes are stored with the account. Each contract account has an ether balance associated with it. The contract code of these accounts gets activated every time a transaction from an EOA or a message from another contract is received by it. When the contract code activates, it allows to read/write the message to the local storage, send messages and create contracts.

How Does Ethereum Work?

Ethereum implements an execution environment called Ethereum Virtual Machine (EVM).

- When a transaction triggers a smart contract all the nodes of the network will execute every instruction.
- All the nodes will run The EVM as part of the block verification, where the nodes will go through the transactions listed in the block and runs the code as triggered by the transaction in the EVM.
- All the nodes on the network must perform the same calculations for keeping their ledgers in sync.
- Every transaction must include:
- Gas limit.
- Transaction Fee that the sender is willing to pay for the transaction.

- If the total amount of gas needed to process the transaction is less than or equal to the gas limit then the transaction will be processed and if the total amount of the gas needed is more than the gas limit then the transaction will not be processed the fees are still lost.
- Thus it is safe to send transactions with the gas limit above the estimate to increase the chances of getting it processed.

Real-World Applications of Ethereum

- **Voting:** Voting systems are adopting Ethereum. The results of polls are available publicly, ensuring a transparent fair system thus eliminating voting malpractices.
- **Agreements:** With Ethereum smart contracts, agreements and contracts can be maintained and executed without any alteration. Ethereum can be used for creating smart contracts and for digitally recording transactions based on them.
- **Banking systems:** Due to the decentralized nature of the Ethereum blockchain it becomes challenging for hackers to gain unauthorized access to the network. It also makes payments on the Ethereum network secure, so banks are using Ethereum as a channel for making payments.
- **Shipping:** Ethereum provides a tracking framework that helps with the tracking of cargo and prevents goods from being misplaced.
- **Crowdfunding:** Applying Ethereum smart contracts to blockchain-based crowdfunding platforms helps to increase trust and information symmetry. It creates many possibilities for startups by raising funds to create their own digital cryptocurrency.
- **Domain names:** Ethereum name service allows crypto users to buy and manage their own domain names on Ethereum, thus simplifying decentralized transactions without putting users to remember long, machine-readable addresses.

A Smart Contract (or cryptocontract) is a computer program that directly and automatically controls the transfer of digital assets between the parties under certain conditions. A smart contract works in the same way as a traditional contract while also automatically enforcing the contract. Smart contracts are programs that execute exactly as they are set up(coded, programmed) by their creators. Just like a traditional contract is enforceable by law, smart contracts are enforceable by code.

- The bitcoin network was the first to use some sort of smart contract by using them to transfer value from one person to another.
- The smart contract involved employs basic conditions like checking if the amount of value to transfer is actually available in the sender account.
- Later, the Ethereum platform emerged which was considered more powerful, precisely because the developers/programmers could make custom contracts in a Turing-complete language.
- It is to be noted that the contracts written in the case of the bitcoin network were written in a Turing-incomplete language, restricting the potential of smart contracts implementation in the bitcoin network.
- There are some common smart contract platforms like Ethereum, Solana, Polkadot, Hyperledger fabric, etc.

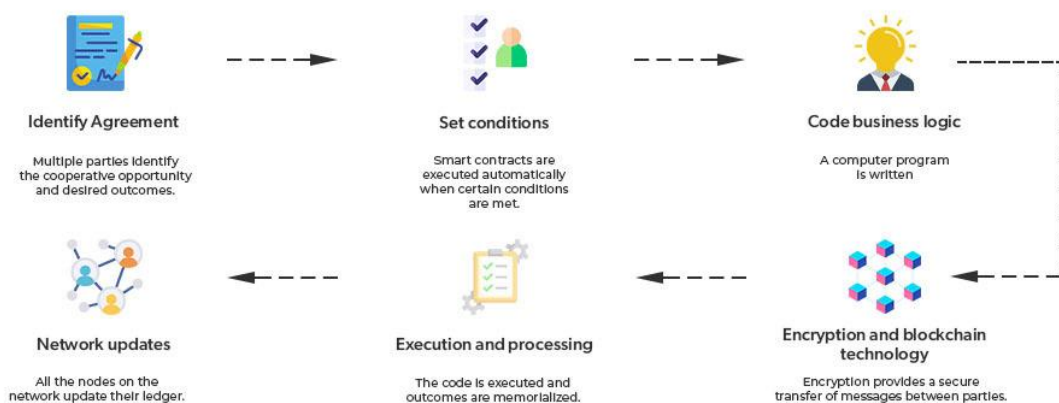
How Do Smart Contracts Work?

The idea behind smart contracts is pretty simple. They are executed on a basis of simple logic, IF-THEN for example:

- IF you send object A, THEN the sum (of money, in cryptocurrency) will be transferred to you.
- IF you transfer a certain amount of digital assets (cryptocurrency, for example, ether, bitcoin), THEN the A object will be transferred to you.
- IF I finish the work, THEN the digital assets mentioned in the contract will be transferred to me.

Note: The WHEN constraint can be added to include the time factor in the smart contracts. It can be seen that these smart contracts help set conditions that have to be fulfilled for the terms of the contract agreement to be executed. There is no limit on how much IF or THEN you can include in your intelligent contract.

How does a Smart Contract Work?



- **Identify Agreement:** Multiple parties identify the cooperative opportunity and desired outcomes and agreements could include business processes, asset swaps, etc.
- **Set conditions:** Smart contracts could be initiated by parties themselves or when certain conditions are met like financial market indices, events like GPS locations, etc.
- **Code business logic:** A computer program is written that will be executed automatically when the conditional parameters are met.
- **Encryption and blockchain technology:** Encryption provides secure authentication and transfer of messages between parties relating to smart contracts.
- **Execution and processing:** In blockchain iteration, whenever consensus is reached between the parties regarding authentication and verification then the code is executed and the outcomes are memorialized for compliance and verification.
- **Network updates:** After smart contracts are executed, all the nodes on the network update their ledger to reflect the new state. Once the record is posted and verified on the blockchain network, it cannot be modified, it is in append mode only.

2. The Turing Completeness of Smart Contract Languages and Verification Challenges

Turing completeness is a concept from computer science that describes a system's ability to perform any computation that can be expressed algorithmically, given enough time and resources. In the context of Ethereum, a Turing-complete system means that its programming language, Solidity, can execute any computational task or algorithm. This capability allows developers to create complex smart contracts and decentralized applications (dApps) with a wide range of functionalities. Essentially, Ethereum's Turing-completeness provides flexibility and power, enabling a diverse array of innovations on its blockchain.

Turing-completeness is a concept from theoretical computer science that refers to a system's capability to perform any computation that can be described by an algorithm. The term is named after the mathematician and computer scientist Alan Turing, who formalized the idea in the 1930s.

1. **Memory:** The ability to store and retrieve data.
2. **Conditional Logic:** The capability to execute different instructions based on certain conditions (e.g., if-then-else statements).
3. **Loops:** The ability to repeat instructions (e.g., while or for loops), which enables the system to perform repetitive tasks.

Turing-Completeness in Blockchain Technology

Turing-completeness in blockchain technology refers to the ability of a blockchain's smart contract platform to support complex computations and logic, similar to how a general-purpose computer can execute any algorithm. This capability significantly expands what can be achieved on the blockchain beyond simple transactions.

1. **Memory and Storage:** The ability to store and manipulate data as needed for complex computations.
2. **Conditional Statements:** Support for conditional logic (e.g., if-else statements) to make decisions based on various inputs.
3. **Loops and Iteration:** The capability to execute repetitive tasks or loops, which are essential for many algorithms and processes.

Advantages of Turing-Completeness in Ethereum

1. **Enhanced Flexibility:** Developers can create smart contracts with intricate logic and conditions, enabling a broad spectrum of functionalities from simple token transfers to complex financial instruments.
2. **Support for Diverse Applications:** Turing-completeness enables the creation of sophisticated DeFi applications, such as decentralized exchanges, lending platforms, and synthetic assets, which rely on complex interactions and calculations.
3. **Innovation and Development Opportunities:** Turing-complete contracts can interact with other contracts and dApps, fostering a rich ecosystem of interconnected services and enabling complex use cases that leverage multiple components.

4. **Automation and Efficiency:** Automation through smart contracts can streamline processes and reduce operational costs by removing intermediaries and reducing administrative overhead.
5. **Enhanced Developer Empowerment:** The Turing-complete nature of Ethereum has led to the development of a robust ecosystem of tools, libraries, and frameworks that support various aspects of smart contract and dApp development.

Challenges of Turing-Completeness in Ethereum

Turing-completeness in Ethereum opens up a vast range of possibilities for decentralized applications and smart contracts but also introduces several challenges that need to be managed carefully.

1. **High Gas Costs:** Executing complex smart contracts often requires more computational resources, leading to higher gas fees. This can make transactions expensive, especially when dealing with intricate contracts or during periods of high network demand.
2. **Network Congestion:** Complex contracts can contribute to network congestion by consuming significant resources, which affects overall transaction speed and network performance.
3. **Difficult Development:** Writing, testing, and debugging Turing-complete smart contracts is inherently more complex compared to simpler systems. Ensuring that contracts behave as expected requires rigorous testing and can be resource-intensive.
4. **Ongoing Maintenance:** Managing and updating smart contracts can be challenging. Changes to the code need to be made carefully to avoid introducing new issues, and once deployed, smart contracts are often immutable, making fixes more complicated.
5. **Limited Throughput:** The computational demands of Turing-complete contracts can limit the Ethereum network's transaction throughput. As more complex contracts and dApps are deployed, they can strain the network's capacity and impact scalability.

Verification Challenges

The flexibility of Turing completeness introduces profound verification difficulties because it is impossible to perfectly predict a program's behaviour without running it (the Halting Problem).

- **Undecidability:** Because of the Halting Problem, static analysis tools cannot guarantee that a smart contract will terminate or behave correctly under all conditions.
- **Reentrancy and State Changes:** Turing-complete contracts often hold complex states. Attackers can call functions repeatedly before the first call finishes, draining funds (e.g., the infamous 2016 DAO attack).
- **Gas Exhaustion/Dos:** An infinite loop, if allowed to run, would freeze network nodes. Ethereum mitigates this with a "gas" fee mechanism, but this creates a "Verifier's Dilemma," where honest nodes are disincentivized to verify complex, high-gas transactions.
- **Immutability:** Once deployed, buggy code is extremely hard to change, making post-deployment fixes nearly impossible.

3. Using smart contracts to enforce legal contracts

Smart contracts are self-executing computer programs on a blockchain that automatically enforce agreements when predefined conditions are met. They eliminate intermediaries, ensuring trustless

execution of terms—like releasing payments upon verified delivery—thereby reducing enforcement costs. While effective for automated tasks, they are best used within, not in place of, formal legal contracts. On a blockchain, a smart contract is a transaction-based program controlled by its predefined code, not by a user. It operates without a need for intermediaries to oversee transactions.

Smart contracts on Bitcoin have historically faced limitations due to the limited scripting language of Bitcoin Script. Bitcoin's creator, Satoshi Nakamoto, deliberately chose this limited language to reduce the risk of errors associated with more complex programming languages.

Despite these constraints, Bitcoin does support basic smart-contract features. Also, Bitcoin protocol upgrades, such as Taproot, have bolstered this blockchain's smart-contract capabilities, among other improvements. Moreover, Bitcoin scaling technologies like the Lightning Network leverage existing scripting functionalities to enable smart contracts for fast and cheap transactions.

Key Aspects of Smart Contract Enforcement

- **Automation:** They use "if-then" logic to immediately execute actions, such as transferring funds or ownership of digital assets.
- **Immutability:** Once deployed on a blockchain terms cannot be changed, providing a permanent, tamper-proof record of the agreement.
- **Legal Standing:** For a smart contract to be legally binding, it must meet traditional contract law requirements (offer, acceptance, consideration).
- **Limitations:** Smart contracts struggle with complex, subjective terms that require human interpretation, making them ideal for objective transactions (e.g., payments, escrow).

4. Comparing Bitcoin scripting vs. Ethereum Smart Contracts

Basis	Bitcoin	Ethereum
Definition	Bitcoin (abbreviation: BTC; sign: ₿) is a decentralized digital currency that can be transferred on the peer-to-peer bitcoin network.	Ethereum is a decentralized global software platform powered by blockchain technology. It is most commonly known for its native cryptocurrency, ether (ETH).
History	The word bitcoin was defined in a white paper published on 31 October 2008. The currency began use in 2009.	Ethereum was conceived in 2013 by programmer Vitalik Buterin, and then went live on 30 July 2015.
Purpose	The purpose of bitcoin was to replace national currencies during the financial crisis of 2008.	The purpose of Ethereum was to utilize blockchain technology for maintaining a decentralized payment network and storing computer code.
Smart Contracts	Although bitcoin do have smart contracts, they are not as flexible or complete as Ethereum smart contracts. Smart contracts in Bitcoin does not have all the functionality that a programming language would give them.	Ethereum allows us to create smart contracts. Smart contracts are computer codes that is stored on a blockchain and executed when the predetermined terms and conditions are met.
Smart Contract Programming Language	Smart contracts on Bitcoin are written in programming languages like Script, Clarity.	Smart contracts on Ethereum are written in programming languages like Solidity, Vyper, etc.

Transactions	Generally, bitcoin transactions are only for keeping notes.	Ethereum transactions may contain some executable code.
Hash Algorithm	Bitcoin runs on the SHA-256 hash algorithm.	Ethereum runs on the Keccak-256 hash algorithm.
Consensus Mechanism	The Proof-of-Work (PoW) is the consensus mechanism used by the Bitcoin network.	The Proof-of-Stake is the consensus mechanism used by Ethereum.
Block Time	The block time of bitcoin is 10 minutes.	The block time of Ethereum is 14 to 15 seconds.
Block Limit	The bitcoin blockchain has a block limit of 1 MB.	The Ethereum blockchain does not have a block limit.
Popularity	Bitcoin is the most popular digital currency in the market to date.	Ether, native currency of Ethereum is the second-largest cryptocurrency after bitcoin to date.
Energy Consumption	Energy consumption is very high.	Energy consumption is very low as compared to bitcoin
Energy Consumption rate	Energy consumption rate of bitcoin mining system 3.2 Million household.	Energy consumption rate of bitcoin mining system 1.2 Million household.
Structure	Structure of bitcoin is simple and robust.	Structure of Ethereum is complex and feature rich
Rewards	Miner got nearly 6.25 BTC on successfully adding new block in network.	Miner got nearly 5 BTC along with same additional rewards on successfully adding new block in network.
Assets	Assets of Bitcoin is BTC.	Assets of Ethereum is Ether.